



# UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



## FCFM

### **Análisis Numérico**

**M.A. Maria Del Carmen Martínez Cejudo**

### **Proyecto 1 Método Montante**

**Sebastian Calderón Carrillo 2087472**

**Edson Armando Amaya Macias 2151976**

**Grupo 031**

## Introducción

El método de Montante, también conocido como algoritmo de Bareiss, es una variante de la eliminación gaussiana que permite resolver sistemas de ecuaciones lineales, así como calcular el determinante, la matriz adjunta y la matriz inversa de una matriz cuadrada, utilizando aritmética exacta con fracciones. A diferencia de la eliminación gaussiana tradicional, en cada etapa el resultado de las operaciones se divide entre el pivote de la etapa anterior, lo que evita la acumulación de errores de redondeo propios de la aritmética en punto flotante. El determinante se obtiene directamente del último pivote calculado, la adjunta se construye a partir de la traspuesta de la matriz de cofactores (cada uno como el determinante de la submatriz menor correspondiente) y la inversa se calcula dividiendo la adjunta entre el determinante.

Para la implementación en Python se dividió el proyecto en módulos con responsabilidades separadas: `modules/montante.py` concentra la lógica matemática (eliminación de Montante, determinante, adjunta, inversa y verificación de solvencia), `modules/utils.py` se encarga de la validación y lectura de datos por consola, y `modules/menu.py` implementa el menú interactivo que conecta ambos módulos. Como decisión de diseño clave, todos los valores numéricos se manejan con el tipo `Fraction` de la librería estándar en lugar de `float`, de modo que los resultados se mantienen exactos durante todo el proceso. El núcleo del programa es la función `eliminacion_montante`, que aplica la fórmula recursiva del método ( $a_{ij} = (pivote \cdot a_{ij} - a_{ik} \cdot a_{kj}) / pivote\_anterior$ ) sobre la matriz aumentada, apoyada en una búsqueda de pivote que intercambia filas cuando el elemento de la diagonal es cero. A partir de esa base se construyeron `determinante` (controlando además el cambio de signo al intercambiar filas), `resolver_sistema` (que arma la matriz aumentada y divide la última columna entre el pivote final) y, apoyándose en `determinante`, las funciones `adjunta` e `inversa`. Antes de resolver cualquier sistema, `resolver_sistema` aplica el teorema de Rouché-Frobenius comparando rangos, de manera que el programa nunca intenta resolver un sistema incompatible o indeterminado. Toda la entrada del usuario se valida con manejo de excepciones en `utils.py`, pidiendo nuevamente el dato en caso de error.

## Codificación

En esta sección se presentan y explican las capturas de pantalla de la lógica modular implementada en el código, destacando cómo el código traduce las propiedades matemáticas del método.

### Eliminación Montante

A continuación se observa la función encargada de ejecutar la pivotación completa en la matriz aumentada.

```
def eliminacion_montante(aumentada):
    """Aplica el metodo de eliminacion Montante a la matriz aumentada.

    ValueError si algun pivote es 0 (sistema singular).
    """
    matriz = _copiar(aumentada)
    n = len(matriz)
    columnas = len(matriz[0]) if n else 0
    pivote_anterior = Fraction(1)
    for k in range(n):
        if not _hallar_pivote(matriz, k):
            raise ValueError(
                "El sistema es singular: pivote igual a cero en la etapa " f"{k + 1}"
            )
        pivote = matriz[k][k]
        for i in range(n):
            if i == k:
                continue
            for j in range(columnas):
                if j == k:
                    continue
                matriz[i][j] = (pivote * matriz[i][j] - matriz[i][k] * matriz[k][j]) / pivote_anterior
            pivote_anterior = pivote
    return matriz
```

### Explicación

La función realiza un ciclo externo sobre cada pivote  $k$ . Mantiene la variable “pivote\_anterior” inicializada en 1. En cada iteración actualiza las celdas que no pertenecen a la fila o columna del pivote actual aplicando el determinante  $2 \times 2$  y dividiendo estrictamente entre “pivote\_anterior”, asegurando divisiones enteras exactas.

## Cálculo del Determinante

La siguiente captura muestra el cálculo optimizado del determinante mediante reducción Montante.

```
def determinante(matriz):
    """Calcular el determinantes usando el metodo Montante."""
    n = len(matriz)
    if n == 0:
        return Fraction(0)
    matriz_temp = _copiar(matriz)
    signo = 1
    pivote_anterior = Fraction(1)
    for k in range(n - 1):
        if matriz_temp[k][k] == 0:
            matriz_temp_cambiada = _hallar_pivote(matriz_temp, k)
            if not matriz_temp_cambiada:
                return Fraction(0)
            signo = -signo
            pivote = matriz_temp[k][k]
            for i in range(k + 1, n):
                for j in range(k + 1, n):
                    matriz_temp[i][j] = (pivote * matriz_temp[i][j] - matriz_temp[i][k] * matriz_temp[k][j]) / pivote_anterior
            pivote_anterior = pivote
    return signo * matriz_temp[n - 1][n - 1]
```

## Explicación

A diferencia de la eliminación completa, solo se reduce la parte inferior de la matriz (submatriz triangular). El último elemento diagonal de la etapa n-1 multiplicada por el acumulador de `signo` (modificado en cada pivoteo con cambio de fila) entrega el determinante exacto.

## Matriz Adjunta e Inversa (`adjunta` e `inversa`)

Se muestran las funciones auxiliares para el cálculo de la adjunta y la inversa.

```
def adjunta(matriz):
    """Obtiene la adjunta de la matriz (traspuesta de la matriz de cofactores)."""
    n = len(matriz)
    if n == 1:
        return [[Fraction(1)]]
    adj = [[Fraction(0)] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            cofactor = determinante(_menor(matriz, i, j))
            if (i + j) % 2 == 1:
                cofactor = -cofactor
            adj[j][i] = cofactor
    return adj

def inversa(matriz):
    """Obtiene la matriz inversa usando adjunta/determinante.

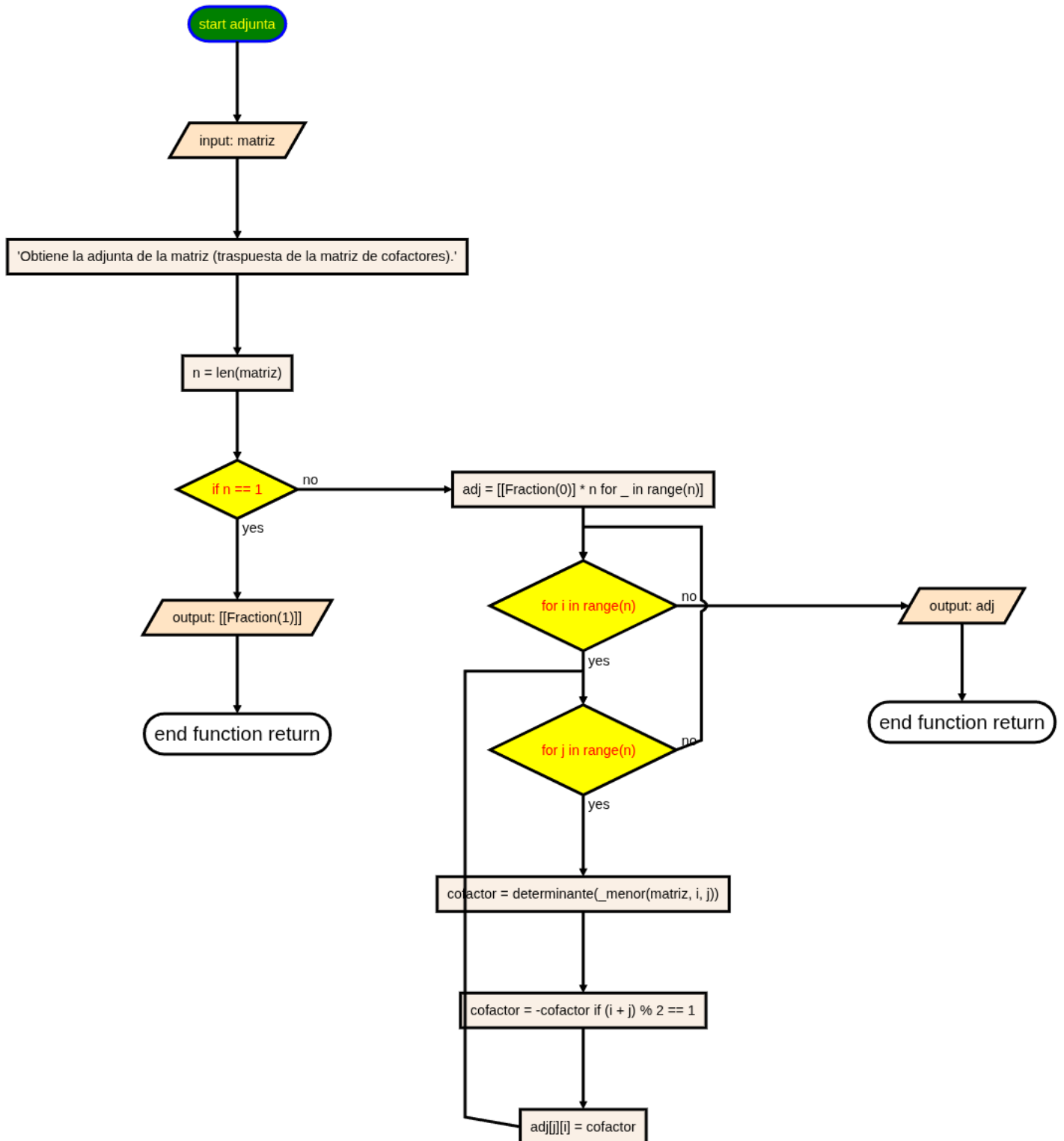
    ValueError si la matriz es singular.
    """
    det = determinante(matriz)
    if det == 0:
        raise ValueError(
            "La matriz no tiene inversa: determinante igual a cero"
        )
    adj = adjunta(matriz)
    n = len(matriz)
    return [[adj[i][j] / det for j in range(n)] for i in range(n)]
```

### Explicación

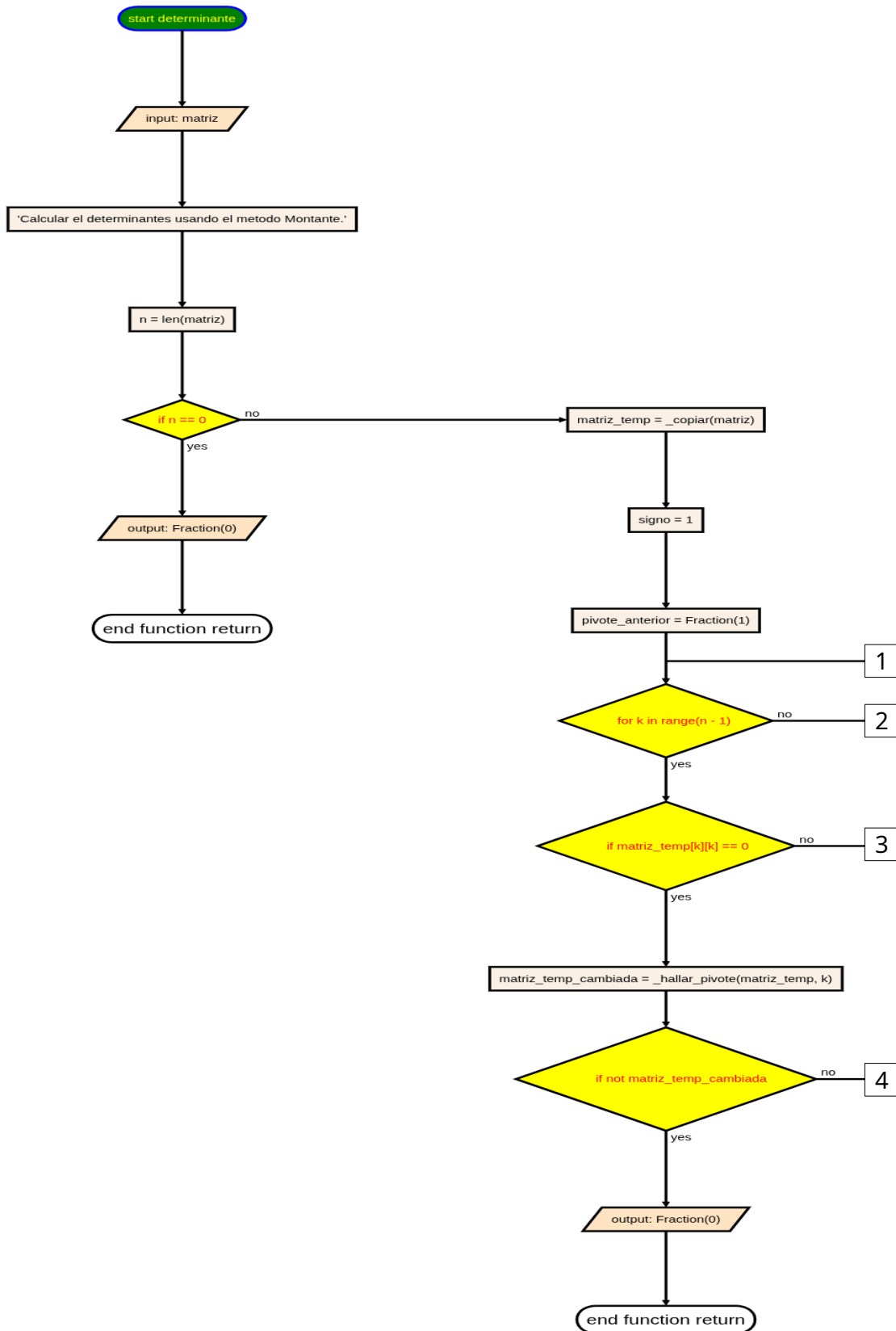
La función "adjunta()" calcula la matriz traspuesta de cofactores mediante menores complementarios obtenidos por el mismo método de Montante. Posteriormente, "inversa()" divide cada entrada de la adjunta entre el determinante final.

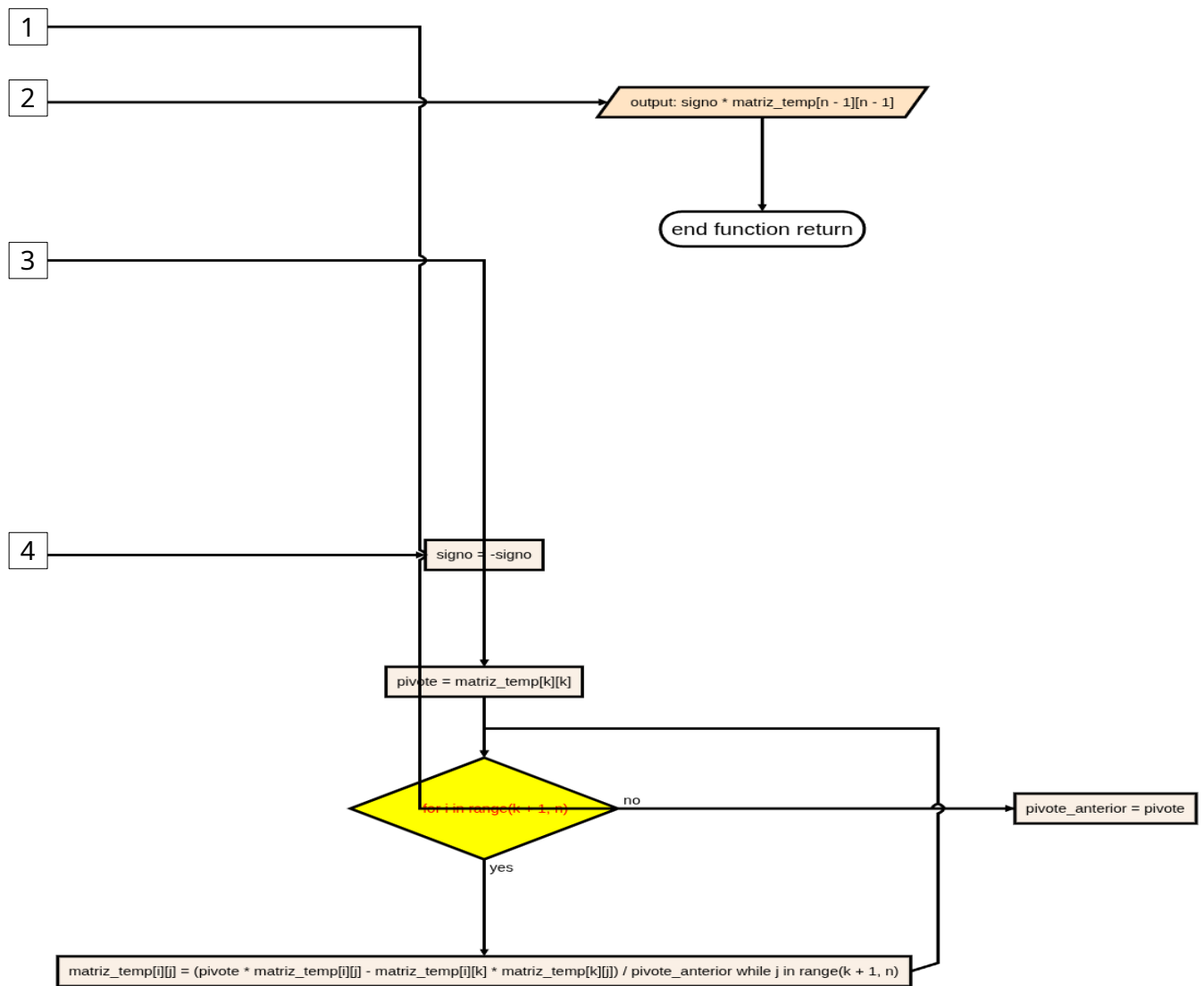
# Diagramas de flujo

## Adjunta

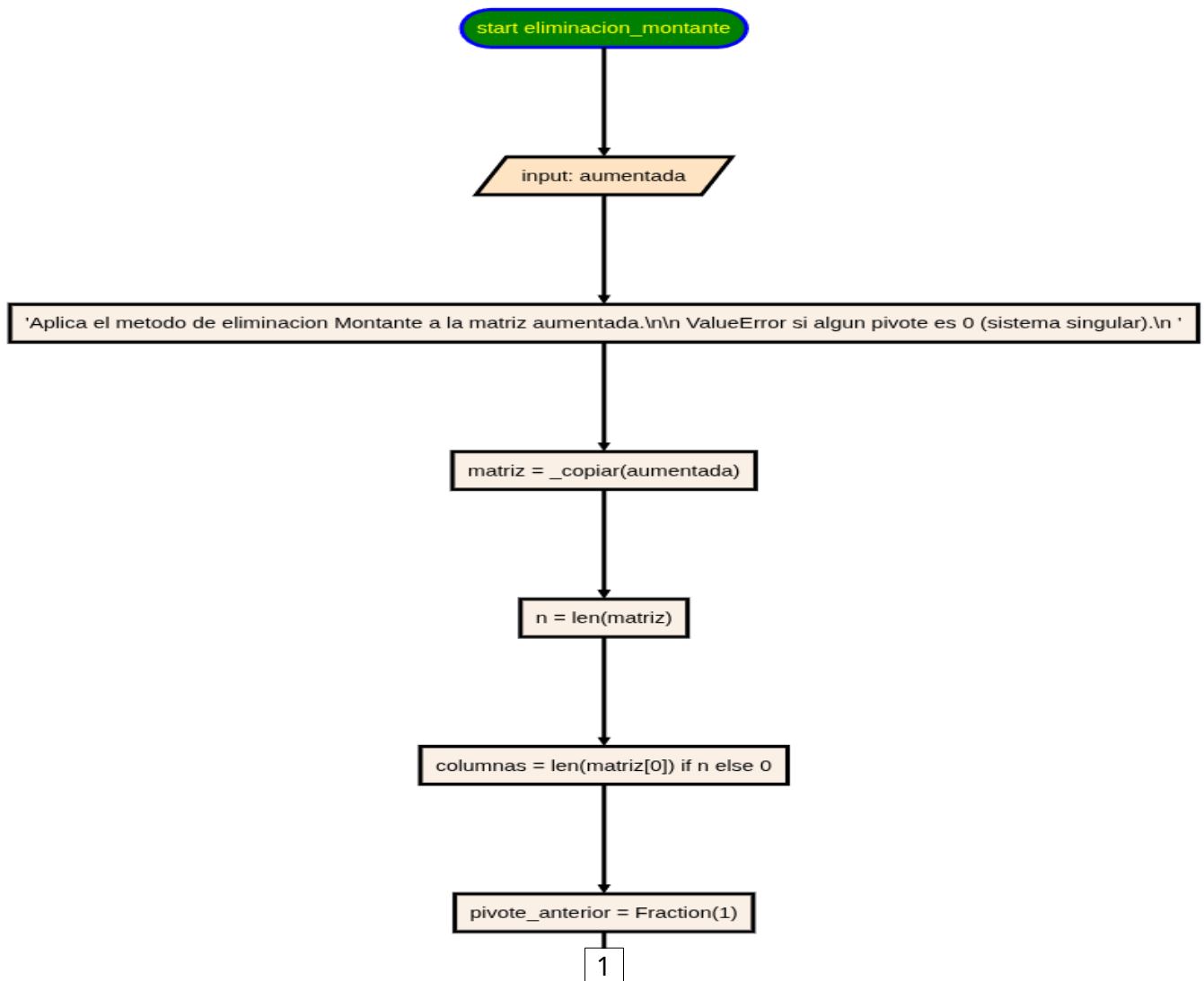


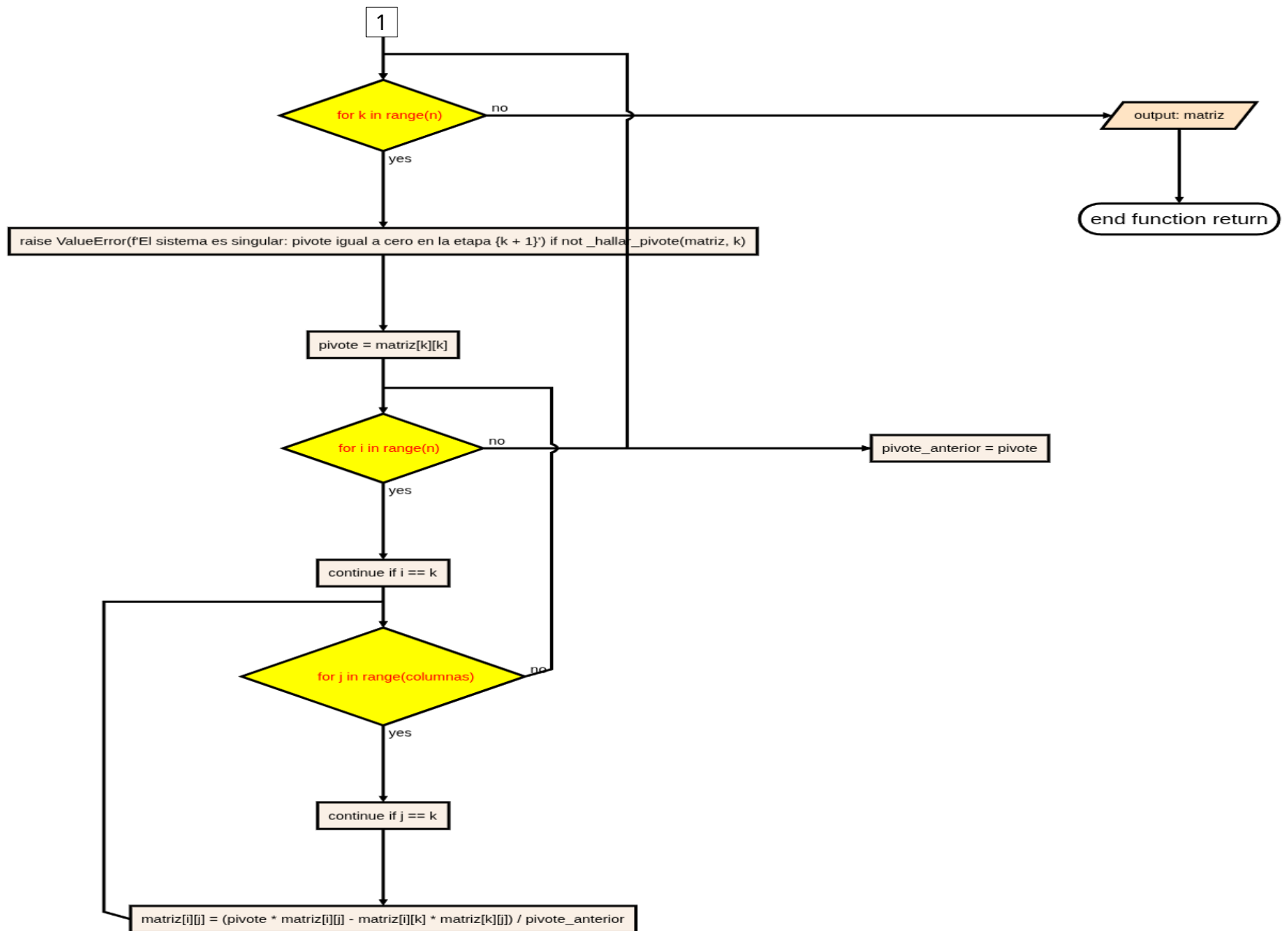
## Determinante



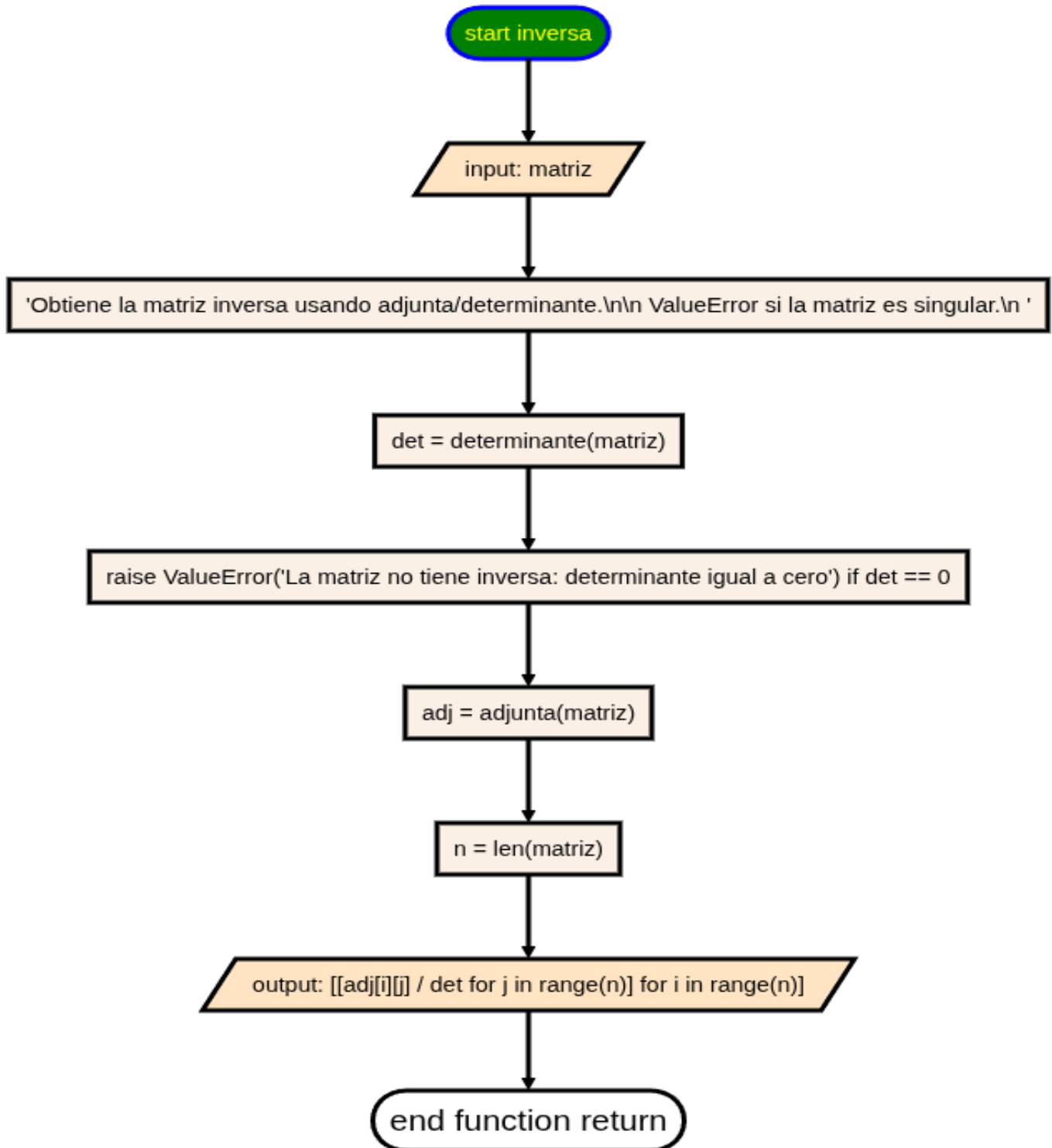


## Eliminación Montante

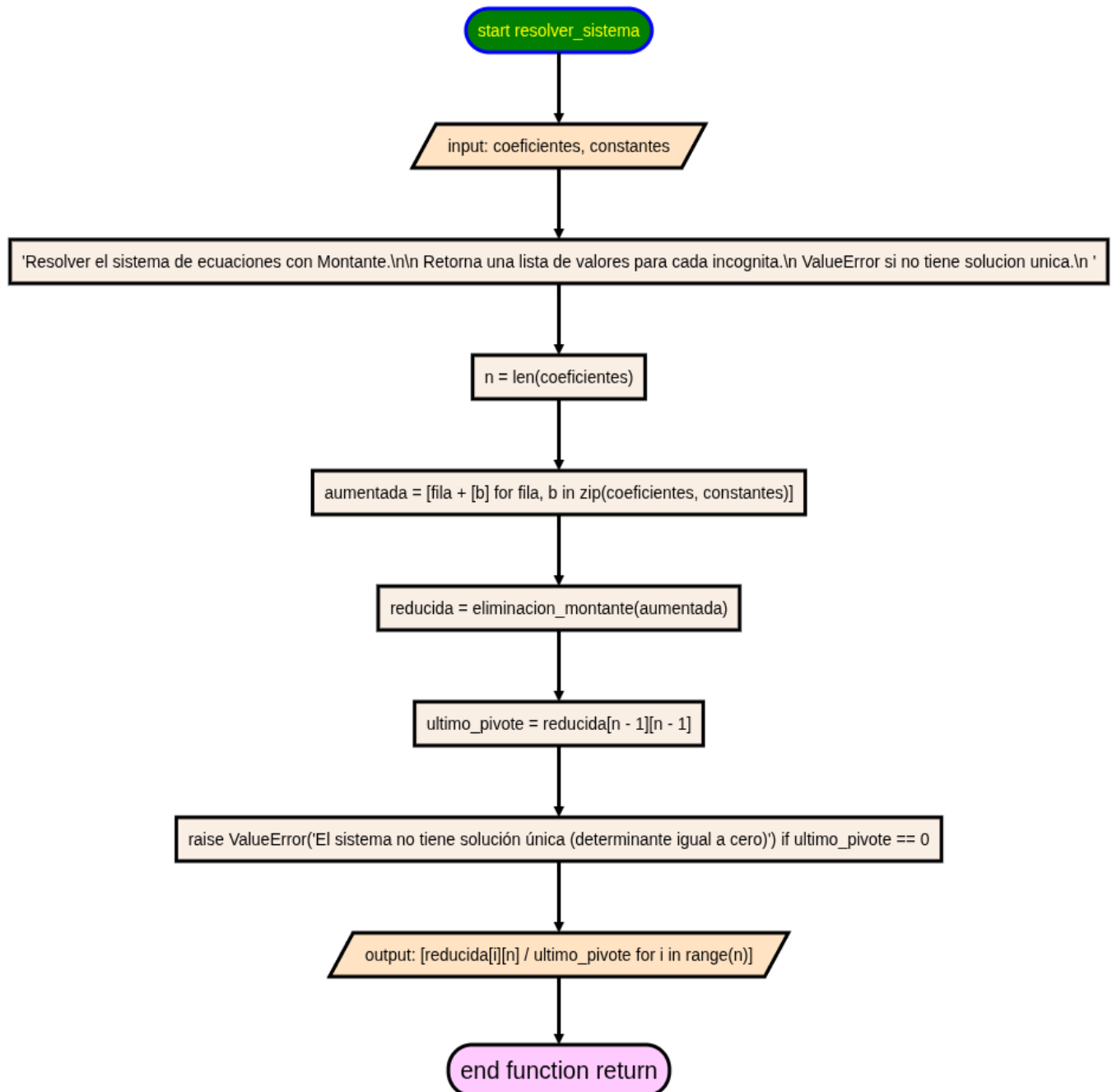




## Inversa



## Resolver Sistema



## Conclusión general

El desarrollo de este proyecto permitió comprobar que el método de Montante es una herramienta eficiente y numéricamente estable para resolver sistemas de ecuaciones lineales, así como para obtener el determinante, la adjunta y la inversa de una matriz, evitando por completo los errores de redondeo gracias al uso de aritmética exacta con fracciones. A nivel de implementación, separar la lógica matemática de la interfaz de usuario facilitó probar cada función por separado y deja un código más fácil de mantener y extender a futuro. Verificar la solvencia del sistema antes de aplicar el método, mediante el teorema de Rouché-Frobenius, resultó fundamental para evitar resultados incorrectos en sistemas incompatibles o indeterminados. En conjunto, el proyecto reforzó tanto el entendimiento teórico del método visto en clase como la capacidad de traducir un algoritmo matemático en código funcional y confiable.

## Ejemplo de ejecución

Para validar el correcto funcionamiento del programa, se propone el siguiente sistema de ecuaciones lineales 3x3:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8 \\ -3x_1 - x_2 + 2x_3 = -11 \\ -2x_1 + x_2 + 2x_3 = -3 \end{cases}$$

```
-- Matriz de coeficientes ---  
  2   1  -1  
-3  -1   2  
-2   1   2  
-- Términos independientes ---  
  8  
-11  
-3
```

$$\det(A) = -1$$

Menú:

- 1) Ingresar / reingresar el sistema de ecuaciones
- 2) Editar matriz o términos independientes
- 3) Obtener resultados de las ecuaciones
- 4) Obtener la matriz adjunta
- 5) Obtener la matriz inversa
- 6) Mostrar el determinante de la matriz
- 7) Mostrar el sistema actual
- 8) Salir

Selecciona una opción (1-8): 6

Determinante de la matriz:  $-1 \approx -1$

$$\text{Adj}(A) = \begin{pmatrix} -4 & -3 & 1 \\ 2 & 2 & -1 \\ -5 & -4 & 1 \end{pmatrix}$$

Menú:

- 1) Ingresar / reingresar el sistema de ecuaciones
- 2) Editar matriz o términos independientes
- 3) Obtener resultados de las ecuaciones
- 4) Obtener la matriz adjunta
- 5) Obtener la matriz inversa
- 6) Mostrar el determinante de la matriz
- 7) Mostrar el sistema actual
- 8) Salir

Selecciona una opción (1-8): 4

--- Matriz adjunta ---

$$\begin{pmatrix} -4 & -3 & 1 \\ 2 & 2 & -1 \\ -5 & -4 & 1 \end{pmatrix}$$

$$A^{-1} = \begin{pmatrix} 4 & 3 & -1 \\ -2 & -2 & 1 \\ 5 & 4 & -1 \end{pmatrix}$$

Menú:

- 1) Ingresar / reingresar el sistema de ecuaciones
- 2) Editar matriz o términos independientes
- 3) Obtener resultados de las ecuaciones
- 4) obtener la matriz adjunta
- 5) obtener la matriz inversa
- 6) Mostrar el determinante de la matriz
- 7) Mostrar el sistema actual
- 8) Salir

selecciona una opción (1-8): 5

--- Matriz inversa ---

```

4    3    -1
-2   -2    1
5     4   -1
```

## Bibliografía

Método de Montante. (2017, June 14). Métodos Numéricos.

<https://metodosnumericos121.wordpress.com/metodo-de-montante/>

Derivando. (2018). El TEOREMA DE ROUCHÉ-FROBENIUS | Una cumbre de las matemáticas escolares [Video]. YouTube. <https://www.youtube.com/watch?v=jnutYsGpKIE>

Rouché–Capelli theorem. (2026). In Wikipedia. [https://en.wikipedia.org/w/index.php?title=Rouch%C3%A9E2%80%93Capelli\\_theorem&oldid=1358871033](https://en.wikipedia.org/w/index.php?title=Rouch%C3%A9E2%80%93Capelli_theorem&oldid=1358871033)